

Projeto MenuFlex – Atividade Somativa

Anthero Franco Sprana

Contextualização

A empresa MenuFlex Tecnologia presta serviços de desenvolvimento de sistemas web para pequenos negócios. Um de seus clientes, o restaurante “Cantina Bella Vita”, solicitou a criação de um sistema para gerenciamento digital de seu cardápio.

A empresa já decidiu que o sistema será desenvolvido em arquitetura web, utilizando API REST para o back-end.

Antes do início do desenvolvimento completo, a equipe técnica precisa validar a modelagem, as decisões tecnológicas e a estrutura da API, garantindo que o projeto seja escalável e organizado.

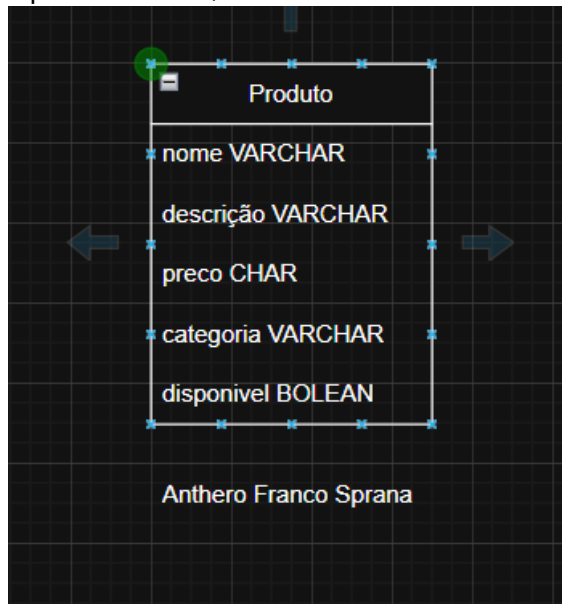
Você foi designado para realizar essa análise técnica inicial.

Etapa A – Modelagem Orientada a Objetos (Diagrama de Classe)

1 – Elabore o **diagrama de classe** da entidade **Produto**, considerando um sistema de cardápio digital.

O diagrama deve conter, no mínimo:

- Atributos e métodos essenciais;
- Tipos de dados;



Etapa B – Persistência e Base de Dados

1 – Explique como a entidade **Produto** seria armazenada em um banco de dados.

Sua resposta deve abordar:

- Tipo de banco de dados mais adequado (relacional ou não relacional);
- Justificativa da escolha;

- Forma de interação entre a API e o banco de dados.

O tipo de Banco de Dados mais adequado seria o relacional, por conta de ele ser mais organizado e as tabelas poderem conversarem entre si. A forma de interação entre a API e o banco de dados é através do arquivo dotenv.

2 - Veja o arquivo database.js:

```
const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'bellavita',
  porta: 3306
})
```

Por que essa conexão não vai funcionar? Explique qual é o erro e como corrigir.

- A conexão não irá funcionar por conta da ausência do requerimento do mysql2 e do dotenv. Para funcionar tem que colocar o "const mysql = require('mysql2') e require('dotenv').config()", depois disso acrescente o process.env em host, user, password, database e porta. Através disso o database poderá processar as informações do arquivo .env que fica salvo os dados. Vale ressaltar que não é porta e sim port.

3 - Observe o código:

```
app.put('/produtos/:id', async (req, res) => {
  const { id } = req.params
  const dados = req.body

  await queryAsync("UPDATE produto SET ? WHERE id = id", [dados])
  res.send("Atualizado")
})
```

Por que esse UPDATE não irá atualizar nenhum produto?

- Por conta que está faltando o try dentro da arrow function, no segundo const, depois de req.params está faltando os dados em si como nome, descricao, preco, categoria, disponível, sem especificar isso o req.body não irá retornar. Está faltando também a validação se o id é um número (isNaN), deve ter a validação de que um o número deve ser positivo, e de que algum dado foi enviado.

Etapas C – Arquitetura da API REST

1 – Descreva a estrutura básica de uma **API REST** para gerenciamento de produtos.

Inclua em sua resposta:

- Endpoints necessários para as operações de CRUD;
- Métodos HTTP associados a cada operação;
- Organização lógica das rotas.

2 – Analise a rota abaixo:

```
app.get('/produtos/id', async (req, res) => {  
  const id = req.params.id  
  const produto = await queryAsync("SELECT * FROM produto WHERE id = ?", [id])  
  res.json(produto)  
})
```

Por que essa rota nunca vai funcionar ao buscar um produto pelo ID?

- Por conta dos dois pontos - : antes do ID.

3 – Desenvolva o método DELETE de produtos.

```
// 5) DELETAR PRODUTO Anthero Franco Sprana
app.delete('/produtos', async (req, res) => {
  try {
    const { nome, descricao, preco, categoria, disponivel } = req.body

    if (!nome || !descricao || !preco) {
      return res.status(400).json({
        sucesso: false,
        mensagem: "Nome, descrição e preço são obrigatórios"
      })
    }

    if (typeof preco !== "number" || preco <= 0) {
      return res.status(400).json({
        sucesso: false,
        mensagem: "Preço deve ser um número positivo"
      })
    }

    const deletarProduto = {
      nome: nome.trim(),
      descricao: descricao.trim(),
      preco,
      categoria: categoria || null,
      disponivel: disponivel ?? true
    }

    const resultado = await queryAsync("DELETE INTO produto SET ?", [deletarProduto])

    res.status(201).json({
      sucesso: true,
      mensagem: "Produto deletado com sucesso",
      id: resultado.insertId
    })

  } catch (erro) {
    res.status(500).json({
      sucesso: false,
      mensagem: "Erro ao deletar produto",
      erro: erro.message
    })
  }
})
})
```

Etapa D – Frameworks e Ambiente de Desenvolvimento

1 – Explique por que o uso do **Node.js com Express** é adequado para o desenvolvimento dessa API.

Sua resposta deve considerar:

- Características do Node.js;
- Papel do framework Express;
- Benefícios para aplicações web.

O framework express é adequado para o desenvolvimento da API por conta de ele ser a “ponte” e poder reduzir o código. Já o node.js é focado no Banco de Dados facilitando para os desenvolvedores.

2 – Descreva, de forma clara e organizada, as etapas iniciais para criar um projeto web em Node.js com persistência de dados em MySQL. Explique também como você estruturaria a organização do projeto (pastas e arquivos principais).

- Primeiramente eu irei no terminal do VScode executar o código *init -y* e depois criar uma pasta para a API - API_MenuFlex, depois dentro dela eu irei instalar o mysql2, dotenv e express através do *npm install*, com isso irá aparecer uma pasta chamada node_modules e os dois arquivos .Json. Depois criarei o arquivo .env e duas subpasta chamada config e src dentro de config terá o arquivo database.js e dentro de src terá os arquivos app.js e server.js.

3 – Para que serve o arquivo .env e por que não devemos subi-lo no github?

- O arquivo .env serve como uma segurança de seus dados do banco, e também o database.js processa os dados através dele, e subindo no GitHub ele ficará exposto para todos, como o User, Password etc.